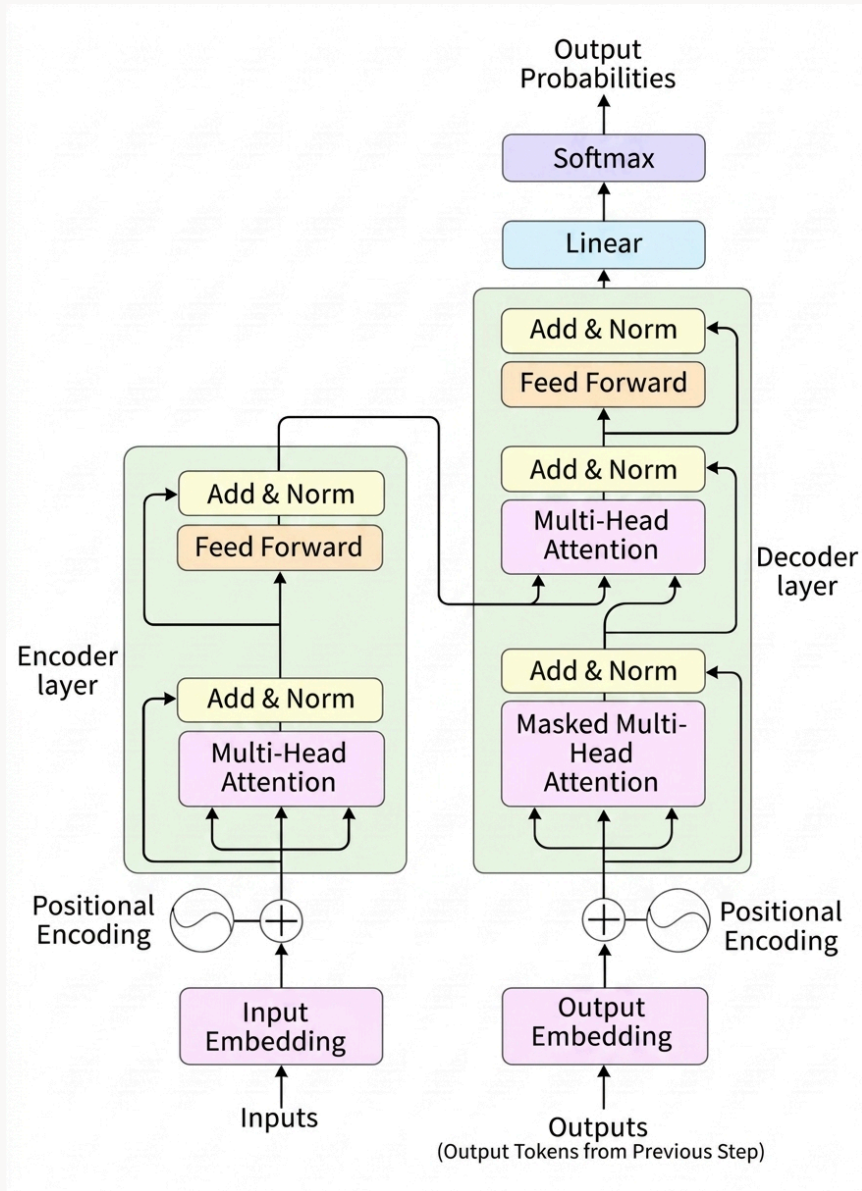


Transformer Theory

A compact walkthrough of attention, masking, position, sampling, and the encoder-decoder stack. Formulas stay next to the reason they exist.



Encoder reads the full input. Decoder writes the next token using masked self-attention plus encoder context.

1. Self-attention / Q, K, V
2. Multi-head attention
3. Masked multi-head attention
4. Positional encoding
5. Cross attention
6. Residual connections
7. Feed-forward network
8. Softmax and output
9. Top-k and top-p
10. Temperature
11. Encoder vs decoder
12. Source and target masks

1. Self-attention

Self-attention lets every token look at every other token in the same sequence. It builds a context-aware vector for each token from Query, Key, and Value vectors.

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

$Q \cdot K^T$

Measures how relevant each token is to every other token.

$\div \sqrt{d_k}$

Stops scores from exploding so softmax stays stable.

$\times V$

Mixes value vectors using those attention weights.

Why it helps

1. Captures relationships between all tokens in a sequence
2. Does not depend on token proximity or distance
3. Uses positional encoding to include order information
4. Works with variable-length sequences

```
import torch
import torch.nn.functional as F

def self_attention(Q, K, V):
    d_k = Q.shape[-1]
    scores = torch.matmul(Q, K.transpose(-2, -1)) / d_k**0.5
    weights = F.softmax(scores, dim=-1)
    output = torch.matmul(weights, V)
    return output, weights
```

Implementation notes

1. QK^T scores pairwise relevance.
2. Dividing by $\sqrt{d_k}$ keeps softmax numerically stable.
3. `F.softmax(scores, dim=-1)` turns scores into probabilities.
4. `torch.matmul(weights, V)` builds the context vector.
5. In transformers, Q, K, and V usually come from learned linear layers on the input embeddings.

2. Multi-head attention

One attention head gives only one way of looking at a sentence. Language carries grammar, nearby words, subject-object links, and meaning at the same time. A single pattern can miss useful context.

Problem

Single-head attention can lock onto one relationship and drop the rest.

Fix

Run several attention operations in parallel. Each head can learn a different relationship.

How it works

- 1 Project input embeddings into multiple Q, K, V sets.
- 2 Each head runs self-attention on its own projections.
- 3 Concatenate the head outputs.
- 4 A final linear layer mixes them together.

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i)$$
$$Q_i = XW_iQ \quad K_i = XW_iK \quad V_i = XW_iV$$
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_n) W^o$$

More heads can mean

- Better coverage of relationships
- Richer context from different views
- More Q, K, V projection matrices
- More parameters, time, and compute

Paper note

The original *Attention Is All You Need* Transformer used **8 heads**. More is not automatically better. Balance quality, cost, and training stability.

MEMORY TRICK

Single-head attention = one attention pattern. Multi-head attention = multiple patterns in parallel.

3. Masked multi-head attention

Masked multi-head attention is multi-head attention that cannot look at some tokens. It lives mainly in the decoder, so the model cannot cheat by reading future words during training.

Why mask?

Text is generated one token at a time. If the model is predicting **machine** in “I love machine learning”, it should see only “I love”.

Autoregressive rule

Each token depends only on previous tokens. Training then matches real generation: left to right, no future leakage.

What the mask allows

Token 1 can see: Token 1

Token 2 can see: Token 1, Token 2

Token 3 can see: Token 1, Token 2, Token 3

Token 4 can see: Token 1, Token 2, Token 3, Token 4

A causal mask is added before softmax. Blocked future scores become a very negative value such as $-\infty$, so their attention weight becomes 0.

$\text{MaskedAttention}(Q, K, V) = \text{softmax}((QK^T / \sqrt{d_k}) + \text{mask}) V$

allowed positions $\rightarrow 0$ blocked future positions $\rightarrow -\infty$

Tiny example: “The cat sat”

1. **The** can see only The
2. **cat** can see The cat
3. **sat** can see The cat sat

The token *cat* cannot look ahead at *sat*.

NORMAL MULTI-HEAD

Tokens may attend to all tokens. Common in the encoder. Good for full input context.

MASKED MULTI-HEAD

Tokens may attend only to past tokens. Common in the decoder. Good for step-by-step generation.

MEMORY TRICK

Multi-head = look in many ways. Masked multi-head = look in many ways, but not into the future.

4. Positional encoding

Transformers do not read tokens in order the way RNNs do. Self-attention sees the whole sequence at once, so the model needs extra information about where each token sits.

Final input = token embedding + positional encoding

Without position, “Dog bites man” and “Man bites dog” look like the same bag of words. Position tells the model which token is first, which is second, and how far tokens are from each other.

Sinusoidal encoding

The original paper uses sine and cosine waves:

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i / d_{\text{model}})})$$
$$\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{(2i / d_{\text{model}})})$$

`pos` is the token index, `i` is the dimension index, `d_model` is the embedding size. Even dimensions use sin. Odd dimensions use cos.

Why sine and cosine?

Repeating waves encode both absolute position and relative distance. Identities such as $\sin(a+b)$ and $\cos(a+b)$ let the model recover offsets with linear transforms.

Even / odd pattern

Dimensions alternate: 0 sin, 1 cos, 2 sin, 3 cos. Each pair shares a wavelength. Every position is a vector of those wave values.

```
token embedding:    [0.6, 0.7, 0.8, 0.9]
positional encoding: [0.0, 1.0, 0.0, 1.0]
final input:        [0.6, 1.7, 0.8, 1.9]
```

Both encoder and decoder need position. The encoder uses it to understand the full sentence. The decoder uses it with masking to generate tokens in order.

MEMORY TRICK

Token embedding = what the word is. Positional encoding = where the word is. Meaning + position = useful Transformer input.

5. Cross attention

Also called encoder-decoder attention or inter-sequence attention. The decoder uses it to look at the encoder output while writing the target sequence.

Setup

Source sequence: I love AI

Target sequence: Nenu AI ni premistunnanu (I love AI)

The decoder should not write blindly. It must align each output token with the input meaning.

Q

Comes from the decoder. The decoder asks the question.

K

Comes from the encoder output.

V

Comes from the encoder output.

$$\text{CrossAttention}(Q_{\text{dec}}, K_{\text{enc}}, V_{\text{enc}}) = \text{softmax}(Q_{\text{dec}} K_{\text{enc}}^T / \sqrt{d_k}) V_{\text{enc}}$$

- 1 Encoder reads the source sentence and emits encoder outputs.
- 2 Decoder starts generating the target sentence.
- 3 Decoder creates Query Q.
- 4 Encoder output supplies Key K and Value V.
- 5 Scores between decoder queries and encoder keys decide which input tokens matter.

SELF-ATTENTION

Q, K, V come from the same sequence. Used inside encoder and decoder. Tokens understand other tokens in the same sentence.

CROSS ATTENTION

Q comes from the decoder. K and V come from the encoder. The decoder looks across to the input while generating.

Decoder order: masked self-attention, then cross attention, then the feed-forward network.

MEMORY TRICK

Self-attention = look inside the same sequence. Cross attention = decoder Q looks at encoder K and V.

6. Residual connections

$$\text{Output} = \text{Layer}(x) + x$$

In Transformer blocks, this shortcut wraps attention layers and feed-forward layers.

Why they exist

- Original input information is preserved
- Gradients flow more easily during backprop
- Training stays more stable
- Deep networks learn faster
- The layer learns the extra improvement, not everything from scratch

Simple idea

Keep the old information, then add useful new information. Residual connection = a shortcut path for information.

7. Feed-forward network

After attention mixes tokens, each token goes through a feed-forward network on its own.

$$\text{FFN}(x) = \text{Linear}_2(\text{Activation}(\text{Linear}_1(x)))$$

Input → Linear → ReLU / GELU → Linear → Output

The FFN adds non-linearity and capacity. It transforms each token representation, learns more complex patterns, and deepens understanding after attention.

MEMORY TRICK

Attention mixes tokens. The feed-forward network improves each token representation.

8. Softmax and output layer

After the decoder produces a final hidden vector, the model must choose the next token.

Decoder output → Linear layer → logits → softmax → probabilities

Output layer

A linear layer maps the decoder vector to vocabulary size. Logits are raw scores. Higher logit means the model thinks that token is more likely.

```
AI: 8.2
ML: 6.5
cat: 1.1
apple: 0.4
```

Softmax

Softmax converts logits into a probability distribution the model can sample from.

```
AI: 0.70
ML: 0.20
cat: 0.07
apple: 0.03
```

MEMORY TRICK

Output layer gives scores. Softmax converts scores into probabilities.

9. Top-k and top-p sampling

These filters decide which candidate tokens the model may choose after softmax.

Top-k

Keep only the **k** most likely tokens. With **k = 3** and probabilities AI 0.40, ML 0.25, data 0.15, cat 0.08, apple 0.05, the model samples only from AI, ML, and data.

Low k is safer. High k is more varied. Very high k can admit weak tokens.

Top-p / nucleus

Keep the smallest set of tokens whose probabilities add up to **p**. With **p = 0.90** and AI 0.40 + ML 0.25 + data 0.15 + model 0.10 = 0.90, the sample set is AI, ML, data, model.

Low p is focused. High p is more diverse. The set size adapts to the distribution.

TOP-K

Fixed number of tokens. Example: always choose from the top 5.

TOP-P

Flexible number of tokens. Keep enough mass to reach probability p.

Decoder output → linear layer → logits → softmax
→ apply top-k / top-p → sample next token

MEMORY TRICK

Top-k = choose from the top k tokens. Top-p = choose from tokens that cover p of the probability mass. Both trade safety against creativity.

10. Temperature

Temperature controls how random or confident next-token choice is. It is applied before softmax.

```
probabilities = softmax(logits / temperature)
```

Low, e.g. 0.2

Sharper distribution. Safer, more focused. Stronger high-probability tokens. Good for facts, code, summaries.

Equal to 1

Normal softmax. Below 1 sharpens. Above 1 flattens.

High, e.g. 1.2

Flatter distribution. More diversity and creativity. More risk of weak or odd tokens.

Temperature changes the *shape* of probabilities. Top-k and top-p decide *which* tokens remain allowed.

```
logits → apply temperature → softmax → top-k / top-p → sample
```

MEMORY TRICK

Temperature controls confidence vs creativity. Low = strict and focused. High = creative and random.

11. Encoder vs decoder

Encoder = reader

- Input tokens
- embeddings + positional encoding
- multi-head self-attention
- Add & Norm
- feed-forward
- Add & Norm
- encoder output

Uses normal self-attention. Every input token can attend to every other input token.

Decoder = writer

- Output tokens shifted right
- embeddings + positional encoding
- masked multi-head self-attention
- Add & Norm
- cross attention to encoder
- Add & Norm
- feed-forward
- Add & Norm
- linear + softmax

Looks at previous output tokens and the encoder output, then emits the next token.

ENCODER

Understands the input. Bidirectional self-attention. Sees the full source sequence.

DECODER

Generates the output. Masked self-attention. Sees past outputs plus encoder context.

MEMORY TRICK

Encoder reads. Decoder writes, using the encoder's understanding.

12. Source mask and target mask

Both masks tell attention which tokens are allowed and which must be ignored.

Source / src mask

Used on the encoder input. Blocks padding tokens so the encoder attends only to real words.

Source: I love AI PAD PAD
Focus: I love AI

Target / tgt mask

Used on the decoder input. Blocks padding *and* future tokens so generation stays left to right.

Predicting “learning”
Allowed: I am
Blocked: transformers

The encoder can see the full real input because it is only understanding. The decoder is writing one token at a time, so looking at future target tokens would leak the answer.

SRC MASK	TGT MASK
Encoder. Ignore padding. Real input tokens may look at all other real input tokens.	Decoder. Ignore padding and future tokens. Keeps next-token training honest.

MEMORY TRICK

Src mask = ignore padding in the input. Tgt mask = ignore padding + do not look into the future.

Reference links

- [The Illustrated Transformer](#)
- [Transformer Explainer](#)
- [TensorFlow Embedding Projector](#)
- [Embedding Atlas examples](#)